

❑ Milestone 6 —Registre automatique des types & Singleton

Sprint V6 ·Al Qalam Stock Manager Thème : métaclasses ·type
·SingletonMeta ·RegistreMouvementMeta ·factory pattern

Objectif

Introduire les **métaclasses** pour résoudre deux problèmes réels de l'application :

1. **Singleton** : garantir qu'il n'existe qu'une **seule instance** de StockService dans toute l'application, quelle que soit l'endroit où on appelle StockService().
 2. **Registre automatique** : faire en sorte que toute nouvelle sous-classe de Mouvement s'**enregistre d'elle-même** dans un registre sans aucune ligne de code supplémentaire —la métaclassse intercepte la définition de la classe.
-

Ce que vous allez construire

```
alqalamv6/
├── metaclasses/                ← NOUVEAU package
│   ├── singleton.py            ← SingletonMeta : surcharge __call__
│   └── registre.py             ← RegistreMouvementMeta : surcharge __init__
├── models/
│   ├── mouvement.py            ← utilise RegistreMouvementMeta + fabriquer()
│   └── types_mouvement.py      ← NOUVEAU : 4 sous-classes auto-
enregistrées
├── services/
│   └── stock_service.py        ← class StockService(metaclass=SingletonMeta)
                                + ajustement_stock() + retour_stock()
└── ui/
    ├── frames/
    │   └── registre_frame.py    ← NOUVEAU : onglet "📖 Registre" + démo Singleton
```

Concepts mis en œuvre

Concept	Fichier	Ce que ça illustre
type est une métaclasse	—(théorie)	<code>type(Produit) → type · type(type) → type</code>
<code>class Meta(type)</code>	singleton.py	Créer sa propre métaclasse en héritant de type
<code>__call__</code> de métaclasse	singleton.py	Intercepte <code>StockService()</code> avant la création
Double-check locking	singleton.py	Singleton thread-safe avec <code>threading.Lock</code>
<code>__init__</code> de métaclasse	registre.py	Intercepte la définition d'une classe
Auto-enregistrement	types_mouvement.py	<code>TYPE_MVT = "entree" → registre</code> sans code explicite
Factory via registre	mouvement.py	<code>Mouvement.fabriquer(type_mvt, ...)</code> → bonne sous-classe
<code>type(cls)</code> depuis une méthode	mouvement.py	Accède à la métaclasse depuis la méthode de classe

La hiérarchie des métaclasses

Objet ordinaire → instance de → Classe → instance de → Métaclasse

`stock (instance)` → instance de → `StockService` → instance de → `SingletonMeta`
`p (Produit)` → instance de → `Produit` → instance de → `type`
`EntreeMouvement()` → instance de → `EntreeMouvement` → instance de → `RegistreMouvement`

```
from services.stock_service import StockService
from models.mouvement import Mouvement
```

```
print(type(StockService))    # <class 'metaclasses.singleton.SingletonMeta'>
print(type(Mouvement))       # <class 'metaclasses.registre.RegistreMouvementMeta'>
print(type(int))             # <class 'type'>
```

Guide de réalisation étape par étape

Étape 1 —Créer SingletonMeta (20 min)

Une métaclasse surcharge les méthodes de type. Pour le Singleton, on surcharge `__call__` car c'est lui qui est invoqué à chaque `StockService()`.

```
# metaclasses/singleton.py
import threading
```

```

class SingletonMeta(type):
    """
    Métaclasse Singleton : garantit une seule instance par classe.

    Normalement : StockService() appelle type.__call__(StockService)
                  → __new__() → __init__() → retourne une nouvelle instance

    Avec SingletonMeta : __call__() vérifie d'abord le registre.
    Si l'instance existe déjà → la retourner directement, sans __new__/__init__.
    """
    _instances = {}          # registre : {classe → instance unique}
    _lock = threading.Lock() # protège la création multi-thread

    def __call__(cls, *args, **kwargs):
        # Vérification rapide SANS Lock (chemin nominal)
        if cls not in SingletonMeta._instances:
            with SingletonMeta._lock:
                # Vérification à nouveau AVEC Lock (évite race condition)
                if cls not in SingletonMeta._instances:
                    # super().__call__() → chemin normal : __new__ + __init__
                    instance = super().__call__(*args, **kwargs)
                    SingletonMeta._instances[cls] = instance
        return SingletonMeta._instances[cls]

Application sur StockService :

# services/stock_service.py
from metaclasses.singleton import SingletonMeta

class StockService(metaclass=SingletonMeta):
    def __init__(self):
        ... # appelé UNE SEULE FOIS, même si StockService() est appelé 100 fois

# Preuve :
a = StockService()
b = StockService()
c = StockService()

assert a is b is c          # True : même objet
assert id(a) == id(b)       # True : même adresse mémoire
print(f"id = {id(a)}")      # toujours la même valeur

Piège à éviter : __init__ n'est appelé qu'à la première instanciation. Les
arguments passés aux appels suivants sont silencieusement ignorés.

```

Étape 2 —Créer RegistreMouvementMeta (20 min)

Pour le registre automatique, on surcharge `__init__` de la métaclasse. Cette méthode est appelée **à la définition de chaque classe**, pas à son instantiation.

```
# metaclasses/registre.py
```

```
class RegistreMouvementMeta(type):
    """
    Métaclasse Registre : auto-enregistrement des sous-classes de Mouvement.

    Quand Python lit :
        class EntreeMouvement(Mouvement):
            TYPE_MVT = "entree"

    IL appelle RegistreMouvementMeta.__init__("EntreeMouvement", (Mouvement,), namespace)
    → On lit TYPE_MVT dans namespace → on l'enregistre dans _registre.
    Aucune ligne de code supplémentaire n'est nécessaire dans EntreeMouvement !
    """
    _registre = {} # { "entree": EntreeMouvement, "sortie": SortieMouvement, ... }

    def __init__(cls, nom, bases, namespace):
        # Toujours déléguer à type.__init__ en premier
        super().__init__(nom, bases, namespace)

        # Auto-enregistrement : si TYPE_MVT est défini et non-None
        # (La classe de base Mouvement a TYPE_MVT = None → pas enregistrée)
        type_mvt = namespace.get("TYPE_MVT")
        if type_mvt is not None:
            RegistreMouvementMeta._registre[type_mvt] = cls

    @classmethod
    def get_classe(mcs, type_mvt):
        return mcs._registre.get(type_mvt)

    @classmethod
    def get_registre(mcs):
        return dict(mcs._registre)
```

Étape 3 —Modifier Mouvement pour utiliser la métaclasse (15 min)

```
# models/mouvement.py
```

```
from datetime import datetime
```

```
from metaclasses.registre import RegistreMouvementMeta
```

```

class Mouvement(metaclass=RegistreMouvementMeta):
    # TYPE_MVT = None → La classe de base n'est PAS enregistrée
    TYPE_MVT = None
    ICONE = "📦"
    LABEL = "Mouvement"
    TYPES_VALIDES = ("entree", "sortie", "ajustement", "retour") # étendu V6

    def __init__(self, ref_produit, type_mvt, qte, note=""):
        if type_mvt not in self.TYPES_VALIDES:
            raise ValueError(f"Type invalide : {type_mvt!r}")
        if qte <= 0:
            raise ValueError(f"Quantité doit être positive : {qte}")
        self.ref_produit = ref_produit
        self.type_mvt = type_mvt
        self.qte = qte
        self.note = note
        self.date = datetime.now().isoformat(timespec="seconds")

    @classmethod
    def fabriquer(cls, type_mvt, ref_produit, qte, note=""):
        """
        [V6] Factory basée sur le registre de la métaclasse.

        type(cls) retourne la métaclasse de Mouvement : RegistreMouvementMeta
        On accède à _registre via la métaclasse pour trouver la bonne classe.

        Mouvement.fabriquer("entree", "CRAY-001", 50)
        → type(Mouvement) = RegistreMouvementMeta
        → RegistreMouvementMeta._registre["entree"] = EntreeMouvement
        → EntreeMouvement("CRAY-001", 50)
        """
        meta = type(cls) # = RegistreMouvementMeta
        classe = meta.get_classe(type_mvt) # cherche dans le registre

        if classe is not None:
            return classe(ref_produit, qte, note) # appelle __init__ de la sous-classe

        return cls(ref_produit, type_mvt, qte, note) # fallback : classe de base

```

Étape 4 —Créer les sous-classes concrètes (15 min)

Chaque sous-classe définit simplement TYPE_MVT. La métaclasse fait le reste.

```

# models/types_mouvement.py
from models.mouvement import Mouvement

```

```

class EntreeMouvement(Mouvement):
    # Python appelle RegistreMouvementMeta.__init__("EntreeMouvement", ...) ici
    # → _registre["entree"] = EntreeMouvement (automatique !)
    TYPE_MVT = "entree"
    ICONE = "📦"
    LABEL = "Entrée stock"

    def __init__(self, ref_produit, qte, note=""):
        super().__init__(ref_produit, "entree", qte, note)

class SortieMouvement(Mouvement):
    TYPE_MVT = "sortie"
    ICONE = "📦"
    LABEL = "Sortie stock"

    def __init__(self, ref_produit, qte, note=""):
        super().__init__(ref_produit, "sortie", qte, note)

class AjustementMouvement(Mouvement):
    """Nouveau V6 : correction d'écart entre stock théorique et réel."""
    TYPE_MVT = "ajustement"
    ICONE = "📦 "
    LABEL = "Ajustement"

    def __init__(self, ref_produit, qte, note=""):
        super().__init__(ref_produit, "ajustement", abs(qte) or 1, note)
        self.delta = qte # peut être négatif (perte) ou positif (gain)

class RetourMouvement(Mouvement):
    """Nouveau V6 : marchandise renvoyée au fournisseur."""
    TYPE_MVT = "retour"
    ICONE = "📦 "
    LABEL = "Retour fournisseur"

    def __init__(self, ref_produit, qte, note=""):
        super().__init__(ref_produit, "retour", qte, note)

```

Vérification en console :

```

import models.types_mouvement # déclenche l'enregistrement des 4 classes
from metaclasses.registre import RegistreMouvementMeta

print(RegistreMouvementMeta.get_registre())

```

```
# {'entree': <class 'EntreeMouvement'>, 'sortie': <class 'SortieMouvement'>,
# 'ajustement': <class 'AjustementMouvement'>, 'retour': <class 'RetourMouvement'>}}

m = Mouvement.fabriquer("ajustement", "PAP-A4", 10, "inventaire annuel")
print(type(m).__name__) # AjustementMouvement
print(m.ICONNE)         # 📄
```

Étape 5 — Étendre StockService avec les nouvelles opérations (20 min)

```
# services/stock_service.py (extraits)
import models.types_mouvement # IMPORTANT : force l'enregistrement avant tout appel

class StockService(metaclass=SingletonMeta):

    @journaliser("entree")
    @valider_qte(min_val=1, max_val=100_000)
    def entree_stock(self, ref, qte, note=""):
        with self._lock:
            ...
            # fabriquer() → consulte Le registre → crée EntreeMouvement
            self._mouvements.append(Mouvement.fabriquer("entree", ref, qte, note))
        self.sauvegarder()

    @journaliser("ajustement")
    def ajustement_stock(self, ref, qte_cible, note=""):
        """
        [V6] Corrige le stock à une quantité absolue (inventaire physique).
        Crée un AjustementMouvement via Le registre.
        """
        with self._lock:
            produit = self._produits[ref]
            qte_avant = produit.qte
            delta = qte_cible - qte_avant
            if qte_cible < 0:
                raise ValueError(f"Quantité cible négative : {qte_cible}")
            produit.qte = qte_cible
            note_auto = note or f"Inventaire : {qte_avant}→{qte_cible} (Δ{delta:+d})"
            self._mouvements.append(
                Mouvement.fabriquer("ajustement", ref, abs(delta) or 1, note_auto)
            )
        self.sauvegarder()

    def stats_par_type(self):
        """[V6] Statistiques des mouvements groupées par type (depuis Le registre)."""
```

```

registre = RegistreMouvementMeta.get_registre()
return {
    type_mvt: {
        "nb" : len([m for m in self._mouvements if m.type_mvt == type_mvt]),
        "qte_totale": sum(m.qte for m in self._mouvements if m.type_mvt == type_mvt),
        "classe" : classe,
        "icone" : classe.ICONE,
        "label" : classe.LABEL,
    }
    for type_mvt, classe in registre.items()
}

```

Étape 6 —Créer l'onglet Registre (25 min)

L'onglet  **Registre** est divisé en deux zones :

Colonne gauche —tableau du registre

Lire le registre depuis la métaclasse

```

registre = RegistreMouvementMeta.get_registre()
stats = stock.stats_par_type()

```

```

for type_mvt, classe in sorted(registre.items()):
    s = stats.get(type_mvt, {"nb": 0, "qte_totale": 0})
    self._arbre.insert("", "end", values=(
        classe.ICONE,          # colonne icône
        type_mvt,              # clé du registre ("entree", "sortie", ...)
        classe.__name__,       # nom Python de la classe
        classe.LABEL,          # libellé lisible
        s["nb"],               # nombre de mouvements de ce type
        s["qte_totale"],       # quantité totale
    ))

```

Colonne droite —démon Singleton

```

def _demo_singleton(self):
    """Appelle StockService() deux fois et compare les id()."""
    from services.stock_service import StockService

    a = StockService()
    b = StockService()

    self._lbl_id_a.configure(text=f"id(a) = {id(a)}")
    self._lbl_id_b.configure(text=f"id(b) = {id(b)}")

    if a is b:

```



```

        self._lbl_egal.configure(text="a is b → True ☐", text_color="green")
    else:
        self._lbl_egal.configure(text="Erreur : instances différentes !", text_color="red")

```

Différence `__call__` vs `__init__` de métaclasse

Création d'une INSTANCE (`StockService()`)

Appelle la MÉTACLASSE `__call__()`
 → `SingletonMeta.__call__(StockService)`

Moment : à l'exécution du code

Utile pour : contrôler les instances

Définition d'une CLASSE

Appelle la MÉTACLASSE `__init__()`
 → `RegistreMouvementMeta.__init__(`
 `"EntreeMouvement",`
 `(Mouvement,),`
 `{"TYPE_MVT": "entree", ...})`

Moment : à la lecture du fichier `.py`

Utile pour : enregistrer les classes

Pourquoi `import models.types_mouvement` est obligatoire

Si on N'importe PAS types_mouvement avant d'appeler fabriquer() :

```

from models.mouvement import Mouvement
m = Mouvement.fabriquer("entree", "CRAY-001", 50)
print(type(m).__name__)  # Mouvement ← fallback ! Le registre est encore vide.

```

Après import :

```

import models.types_mouvement
m = Mouvement.fabriquer("entree", "CRAY-001", 50)
print(type(m).__name__)  # EntreeMouvement ← correct !

```

Les classes sont enregistrées au moment de leur **définition**, c'est-à-dire à l'import du module. `stock_service.py` doit donc importer `models.types_mouvement` dès son chargement.

Critères de validation

- ☐ `py -3 main.py` lance l'app avec 5 onglets sans erreur
- ☐ `StockService()` is `StockService()` retourne `True` (vérifiable dans l'onglet Registre)
- ☐ L'onglet "☐ Registre" affiche les 4 types : entree, sortie, ajustement, retour
- ☐ `Mouvement.fabriquer("entree", ...)` retourne une instance de `EntreeMouvement`

- ☐ `Mouvement.fabriquer("ajustement", ...)` retourne une instance de `AjustementMouvement`
 - ☐ Bouton "□ Ajuster stock" → dialogue avec delta en temps réel → stock modifié
 - ☐ Bouton "□ Retour fourn." → stock diminue → mouvement de type "retour" visible
 - ☐ Onglet Registre → compteur "Nb mvts" augmente après chaque opération
 - ☐ Cliquer "□ Appeler StockService()" N fois → id mémoire toujours identique
 - ☐ `type(StockService)` retourne `SingletonMeta` (console)
 - ☐ `type(Mouvement)` retourne `RegistreMouvementMeta` (console)
-

Pour aller plus loin

- Ajouter un **5ème type** `TransfertMouvement` (transfert entre deux dépôts) : une seule sous-classe à définir, le registre se met à jour automatiquement sans toucher à `StockService`
 - Créer une métaclassse `AbstractMeta` qui lève `TypeError` si on instancie une classe marquée `ABSTRACT = True`
 - Combiner `SingletonMeta` et `RegistreMouvementMeta` en une seule métaclassse via l'héritage de métaclassses
 - Étudier `__init_subclass__` (Python 3.6+) comme alternative légère à la métaclassse pour l'auto-enregistrement —comparer les deux approches
-

Al Qalam Stock Manager —Formation Python Partie II —Sprint V6